SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO3: Articulation** (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Experiments
		Total Marks:	100
CO3 Statement:	Binary Search		
Student Name:	Yaswanthika Shree S	**Reg. No.:**	192565069
Section / Batch:	B.E-CSE-Cyber Security	**Date:**	03.08.2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Write the algorithm and execute the code for the given experiment.
- Follow a well-structured, systematic approach to design and implement an optimal solution.
- Provide insightful analysis with accurate validation, supported by clear documentation including diagrams, code, and results.

Question Type	Focus Area	Questions
Experiments	Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.	Q1

SECTION A – Experiments

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Code of Conduct

I <u>Yaswanthika Shree S</u>(192565069) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yaswanthika Shree S

RUBRICS FOR CALCULATION

Experiments					
Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Understanding of Concepts	25%	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30%	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15%	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10%	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Experimental Design	30%				
Use of Tools	20%				
Analysis of Results	15%				
Documentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: ________________	Signature: ________________	Signature: ________________
Date: ____________________	Date: ____________________	Date: ____________________

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Aim
To compare the performance of **Array-based Binary Search** and **Binary Search Tree (BST) Search** by measuring execution time and analyzing the effect of tree depth on search complexity.

Explanation
Binary Search works on a **sorted array** by repeatedly dividing the search space into two halves until the key is found.
Binary Search Tree (BST) Search compares the key with the root node and recursively searches the left or right subtree. The search performance depends on the **height (depth)** of the tree. A balanced BST performs efficiently, while an unbalanced BST may take longer.

Pseudo Code
START

Read number of elements
Read sorted array
Read search key

Perform Array Binary Search
Record execution time

Create Binary Search Tree
Insert all elements into BST

Perform BST Search
Record execution time

Display search result and execution time
Compare both methods

STOP

Input
7
10 20 30 40 50 60 70
50

Output
Array Binary Search: Found
Execution Time (Array): 0.000006 sec

BST Search: Found
Execution Time (BST): 0.000009 sec

Result
Both Array Binary Search and BST Search successfully found the element. Array Binary Search was slightly faster in this experiment because it directly accesses array elements, while BST search performance depends on the tree height.

SIMATS ENGINEERING

SIMATS Online Programming Lab

DAA PROGRAMMING

Yaswanthika Shree
192565069RC

QUESTIONS

Q1
Binary Search
100 marks

0/1 answered

Q1 Binary Search

100 marks

Analyze Binary Search tree depth versus array-based Binary Search performance experimentally. Implement both approaches and record execution time. Interpret how tree height affects search complexity. Justify differences in performance between tree-based and array-based searches.

Your Code

```
import time

class Node:
   def__init__(self, data):
      self.data = data
      self.left = None
      self.right = None

def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.data:
       root.left = insert(root.left, key)
    elif key > root.data:
        root.right = insert(root.right, key)
    return root

def search_bst(root, key):
    if root is None:
        return False
    if root.data == key:
        return True
    elif key < root.data:
        return search_bst(root.left, key)
    else:
        return search_bst(root.right, key)

def binary_search(arr, key):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return True
        elif arr[mid] < key:
```

Input

```
7
10 20 30 40 50 60 70
50
```

Output

```
Array Binary Search: Found
Execution Time (Array): 0.000006 sec

BST Search: Found
Execution Time (BST): 0.000009 sec
```

Run Save Submit